

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: ARNOLD B. PLATON

Program: DOCTOR OF INFORMATION TECHNOLOGY

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in *Loading_Dataset_and_CNN.ipynb* for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	MANUAL HYPERPARAMETER SELECTION
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	Arnold B. Platon
Date	9/5/2026
Experiment / Trial No.	4

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	<i>e.g. 50 / 80 / 100 / 128 / 150</i>	100	128	Increasing the image size will make the images more finer and the details of both animals will be more identifiable.
Color Mode	<i>Grayscale / RGB</i>	<i>Grayscale</i>	Grayscale	This cuts the input down to one color. Color does not matter in identifying the specific animal since the basis is on its attributes. Color is not needed to tell the shapes apart.
Pixel Normalization	<i>X/255.0 / none</i>	<i>X/255.0</i>	x/255.0	It helps the network's math or the gradient descent to converge faster and more smoothly because it rescales them 0 to 1.
Shuffle Training Data	<i>Yes / No</i>	<i>Yes (random.shuffle)</i>	Random shuffle	Without shuffling, the model would see all dogs, then all cats, in a row and could learn the wrong pattern.

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Held-out Test Set (separate from training)	<i>Yes / No</i>	<i>No (validation_split only)</i>	No	Does not affect the quality of the results.
Validation Split (of training set)	<i>0.0 - 0.3</i>	<i>0.1</i>	0.2	Sets aside more images to check performance on but leaves less to train with.

3. Model Architecture Hyperparameters

TRIAL 1

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	<i>16 / 32 / 64 / 128</i>	<i>64</i>	<i>64</i>	Testing based on the default values	0.8876	0.7885
Conv2D Layer 1 – Kernel Size	<i>(3,3) / (5,5)</i>	<i>(3,3)</i>	<i>(3,3)</i>			
Conv2D Layer 1 – Activation	<i>relu / tanh / elu / leaky_relu</i>	<i>relu</i>	<i>relu</i>			
Pooling Layer 1 – Type & Pool Size	<i>MaxPooling2D / AveragePooling2D; (2,2) / (3,3)</i>	<i>MaxPooling2D, (2,2)</i>	<i>MaxPooling2D, (2,2)</i>			
Dense (Fully Connected) Units	<i>16 / 32 / 64 / 128</i>	<i>64</i>	<i>64</i>			
Dropout Rate	<i>0.0 - 0.5</i>	<i>None (not used)</i>	<i>None (not used)</i>			
Output Layer Activation	<i>sigmoid (binary) / softmax (multi-class)</i>	<i>sigmoid</i>	<i>sigmoid</i>			

TRIAL 2

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	<i>16 / 32 / 64 / 128</i>	<i>64</i>	<i>64</i>		0.9032	0.7915
Conv2D Layer 1 – Kernel Size	<i>(3,3) / (5,5)</i>	<i>(3,3)</i>	<i>(3,3)</i>			

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Activation	<i>relu / tanh / elu / leaky_relu</i>	<i>relu</i>	<i>relu</i>			
Pooling Layer 1 – Type & Pool Size	<i>MaxPooling2D / AveragePooling2D; (2,2) / (3,3)</i>	<i>MaxPooling2D, (2,2)</i>	<i>MaxPooling2D, (2,2)</i>			
Dense (Fully Connected) Units	<i>16 / 32 / 64 / 128</i>	<i>64</i>	<i>64</i>			
Dropout Rate	<i>0.0 - 0.5</i>	<i>None (not used)</i>	<i>None (not used)</i>			
Output Layer Activation	<i>sigmoid (binary) / softmax (multi-class)</i>	<i>sigmoid</i>	<i>sigmoid</i>			

TRIAL 3

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	<i>16 / 32 / 64 / 128</i>	<i>64</i>	<i>64</i>		0.9019	0.8388
Conv2D Layer 1 – Kernel Size	<i>(3,3) / (5,5)</i>	<i>(3,3)</i>	<i>(3,3)</i>			
Conv2D Layer 1 – Activation	<i>relu / tanh / elu / leaky_relu</i>	<i>relu</i>	<i>relu</i>			
Pooling Layer 1 – Type & Pool Size	<i>MaxPooling2D / AveragePooling2D; (2,2) / (3,3)</i>	<i>MaxPooling2D, (2,2)</i>	<i>MaxPooling2D, (3,3)</i>			
Dense (Fully Connected) Units	<i>16 / 32 / 64 / 128</i>	<i>64</i>	<i>32</i>			
Dropout Rate	<i>0.0 - 0.5</i>	<i>None (not used)</i>	<i>None (not used)</i>			
Output Layer Activation	<i>sigmoid (binary) / softmax (multi-class)</i>	<i>sigmoid</i>	<i>sigmoid</i>			

TRIAL 4

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	64	64	N/A	Session Crashes using the Google Colab standard setup	Session Crashes using the Google Colab standard setup
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(5,5)			
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu			
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	(3,3)			
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64			
Dropout Rate	0.0 - 0.5	None (not used)				
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid			

4. Training Hyperparameters

TRIAL 1

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	Based on default	0.8876	0.7885
Optimizer	adam / sgd / rmsprop	adam	adam	Based on default		

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001 (Adam default)	Based on default		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Based on default		
Number of Epochs	e.g. 10 - 200	200	10	Finding the best number of epoch starting with the minimum value set on the range column.		
Early Stopping	Yes (patience = ____) / No	No	No	Based on default		

TRIAL 2

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	Based on default	0.9062	0.7915
Optimizer	adam / sgd / rmsprop	adam	adam	Based on default		
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001 (Adam default)	Based on default		
Batch Size	8 / 16 / 32 / 64 / 128	32	128	Based on default		
Number of Epochs	e.g. 10 - 200	200	15	Finding the best number of epoch starting with the minimum value set on the range column.		
Early Stopping	Yes (patience = ____) / No	No	No	Based on default		

TRIAL 3

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	<i>binary_crossentropy / categorical_crossentropy</i>	<i>binary_crossentropy</i>	<i>binary_crossentropy</i>	Based on default	0.9019	0.8388
Optimizer	<i>adam / sgd / rmsprop</i>	<i>adam</i>	<i>adam</i>	Based on default		
Learning Rate	<i>0.1 / 0.01 / 0.001 / 0.0001</i>	<i>0.001 (Adam default)</i>	<i>0.001 (Adam default)</i>	Based on default		
Batch Size	<i>8 / 16 / 32 / 64 / 128</i>	<i>32</i>	<i>128</i>	Based on default		
Number of Epochs	<i>e.g. 10 - 200</i>	<i>200</i>	<i>15</i>	Finding the best number of epoch starting with the minimum value set on the range column.		

TRIAL 4

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	<i>binary_crossentropy / categorical_crossentropy</i>	<i>binary_crossentropy</i>	<i>binary_crossentropy</i>	N/A	Session Crashes using the Google Colab standard setup	Session Crashes using the Google Colab standard setup
Optimizer	<i>adam / sgd / rmsprop</i>	<i>adam</i>	<i>adam</i>			
Learning Rate	<i>0.1 / 0.01 / 0.001 / 0.0001</i>	<i>0.001 (Adam default)</i>	<i>0.001 (Adam default)</i>			
Batch Size	<i>8 / 16 / 32 / 64 / 128</i>	<i>32</i>	<i>128</i>			
Number of Epochs	<i>e.g. 10 - 200</i>	<i>200</i>	<i>20</i>			
Early Stopping	<i>Yes (patience = ____) / No</i>	<i>No</i>	<i>No</i>			

Early Stopping	Yes (patience = ____) / No	No	No	Based on default		
----------------	-----------------------------	----	----	------------------	--	--

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	0.9019	
Validation Accuracy	0.8388	
Test Accuracy	0.8304 calculated from the confusion matrix	
Test Loss	0.4017	
Precision / Recall / F1 (macro avg)	For Cat (treating Cat as positive): Precision = $1545 / (1545 + 306) = 0.8347$, Recall = $1545 / (1545 + 330) = 0.8240$, $F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$ $F1 = 0.8293$ For Dog (treating Dog as positive): Precision = $1569 / (1569 + 330) = 0.8262$, Recall = $1569 / (1569 + 306) = 0.8368$, $F1 = 0.8315$ Macro average: Precision = 0.8305, Recall = 0.8304, $F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$ $F1 = 0.8304$ Overall accuracy from this matrix (correct predictions divided by total): $(1569 + 1545) / 3750 = 0.8304$, or about 83%.	

5. a. Paste here all screenshots/figures of the tuned model

TRIAL 1

model.summary()

... Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 2)	20
activation (Activation)	(None, 126, 126, 2)	0
max_pooling2d (MaxPooling2D)	(None, 63, 63, 2)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	1,216
activation_1 (Activation)	(None, 61, 61, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 64)	36,928
activation_2 (Activation)	(None, 28, 28, 64)	0
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 64)	0
flatten (Flatten)	(None, 12544)	0
dense (Dense)	(None, 64)	802,880
dense_1 (Dense)	(None, 64)	4,160
dense_2 (Dense)	(None, 1)	65
activation_3 (Activation)	(None, 1)	0

Total params: 845,269 (3.22 MB)
Trainable params: 845,269 (3.22 MB)
Non-trainable params: 0 (0.00 B)

Epoch 1/10

133/133 ————— 264s 2s/step - accuracy: 0.5602 - loss: 0.6857 - val_accuracy: 0.6278 - val_loss: 0.6570

Epoch 2/10

133/133 ————— 259s 2s/step - accuracy: 0.6762 - loss: 0.6072 - val_accuracy: 0.7153 - val_loss: 0.5705

Epoch 3/10

133/133 ————— 271s 2s/step - accuracy: 0.7386 - loss: 0.5365 - val_accuracy: 0.7489 - val_loss: 0.5175

Epoch 4/10

133/133 ————— 263s 2s/step - accuracy: 0.7700 - loss: 0.4817 - val_accuracy: 0.7784 - val_loss: 0.4760

Epoch 5/10

133/133 ————— 267s 2s/step - accuracy: 0.8009 - loss: 0.4356 - val_accuracy: 0.7826 - val_loss: 0.4704

Epoch 6/10

133/133 ————— 256s 2s/step - accuracy: 0.8206 - loss: 0.3999 - val_accuracy: 0.7894 - val_loss: 0.4685

Epoch 7/10

133/133 ————— 260s 2s/step - accuracy: 0.8360 - loss: 0.3649 - val_accuracy: 0.7946 - val_loss: 0.4720

Epoch 8/10

133/133 ————— 254s 2s/step - accuracy: 0.8509 - loss: 0.3392 - val_accuracy: 0.7854 - val_loss: 0.4981

Epoch 9/10

133/133 ————— 260s 2s/step - accuracy: 0.8722 - loss: 0.2963 - val_accuracy: 0.8049 - val_loss: 0.4734

Epoch 10/10

133/133 ————— 261s 2s/step - accuracy: 0.8876 - loss: 0.2628 - val_accuracy: 0.7885 - val_loss: 0.5315

TRIAL 2

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 2)	20
activation (Activation)	(None, 126, 126, 2)	0
max_pooling2d (MaxPooling2D)	(None, 63, 63, 2)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	1,216
activation_1 (Activation)	(None, 61, 61, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 64)	36,928
activation_2 (Activation)	(None, 28, 28, 64)	0
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 64)	0
flatten (Flatten)	(None, 12544)	0
dense (Dense)	(None, 64)	802,880
dense_1 (Dense)	(None, 64)	4,160
dense_2 (Dense)	(None, 1)	65
activation_3 (Activation)	(None, 1)	0

Total params: 2,535,809 (9.67 MB)
 Trainable params: 845,269 (3.22 MB)
 Non-trainable params: 0 (0.00 B)
 Optimizer params: 1,690,540 (6.45 MB)

Epoch 1/15

133/133 ————— 281s 2s/step - accuracy: 0.5567 - loss: 0.6899 - val_accuracy: 0.6473 - val_loss: 0.6274

Epoch 2/15

133/133 ————— 270s 2s/step - accuracy: 0.6487 - loss: 0.6243 - val_accuracy: 0.6680 - val_loss: 0.5999

Epoch 3/15

133/133 ————— 261s 2s/step - accuracy: 0.6882 - loss: 0.5875 - val_accuracy: 0.7108 - val_loss: 0.5594

Epoch 4/15

133/133 ————— 277s 2s/step - accuracy: 0.7162 - loss: 0.5485 - val_accuracy: 0.7289 - val_loss: 0.5229

Epoch 5/15

133/133 ————— 315s 2s/step - accuracy: 0.7517 - loss: 0.5056 - val_accuracy: 0.7621 - val_loss: 0.4911

Epoch 6/15

133/133 ————— 264s 2s/step - accuracy: 0.7841 - loss: 0.4611 - val_accuracy: 0.7786 - val_loss: 0.4669

Epoch 7/15

133/133 ————— 350s 2s/step - accuracy: 0.8018 - loss: 0.4237 - val_accuracy: 0.7972 - val_loss: 0.4454

Epoch 8/15

133/133 ————— 291s 2s/step - accuracy: 0.8267 - loss: 0.3865 - val_accuracy: 0.7882 - val_loss: 0.4697

Epoch 9/15

133/133 ————— 259s 2s/step - accuracy: 0.8411 - loss: 0.3530 - val_accuracy: 0.7972 - val_loss: 0.4393

Epoch 10/15

133/133 ————— 258s 2s/step - accuracy: 0.8525 - loss: 0.3377 - val_accuracy: 0.7911 - val_loss: 0.4720

Epoch 11/15

133/133 ————— 307s 2s/step - accuracy: 0.8653 - loss: 0.3091 - val_accuracy: 0.7936 - val_loss: 0.4863

Epoch 12/15

133/133 ————— 276s 2s/step - accuracy: 0.8808 - loss: 0.2824 - val_accuracy: 0.8066 - val_loss: 0.4751

Epoch 13/15

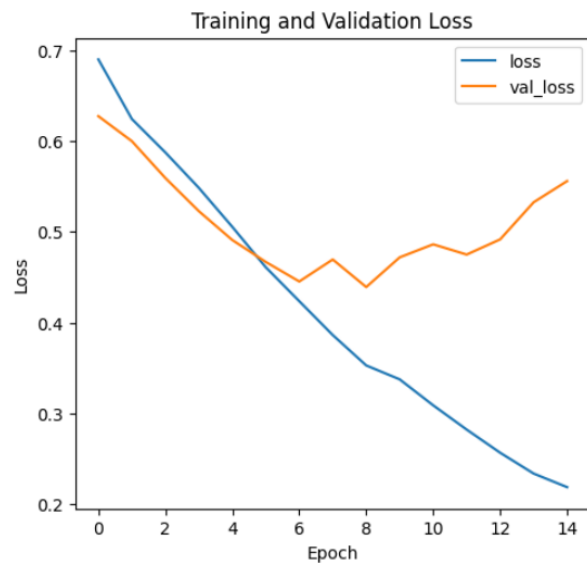
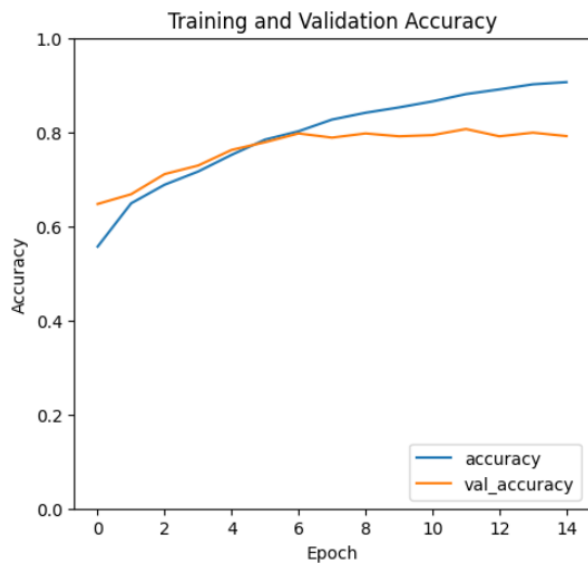
133/133 ————— 264s 2s/step - accuracy: 0.8908 - loss: 0.2569 - val_accuracy: 0.7911 - val_loss: 0.4918

Epoch 14/15

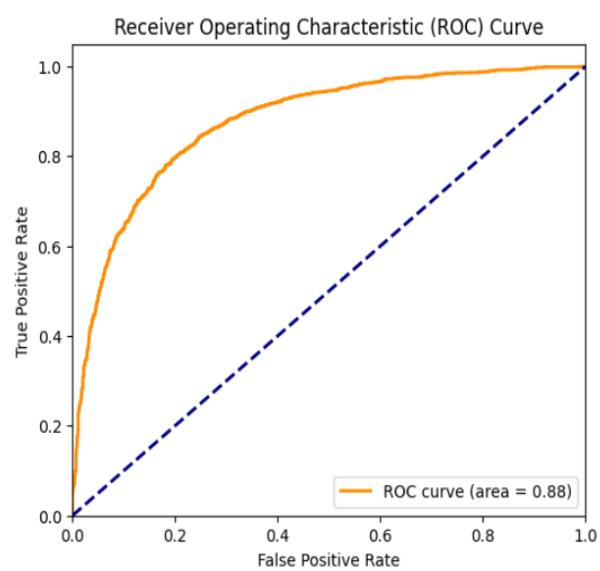
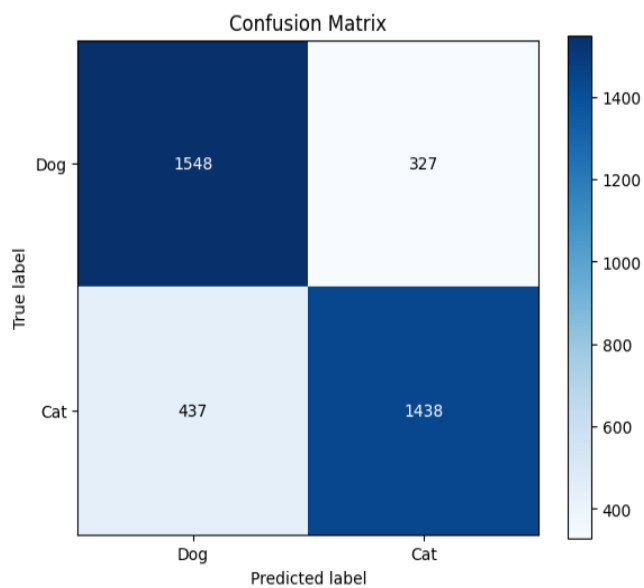
133/133 ————— 260s 2s/step - accuracy: 0.9016 - loss: 0.2340 - val_accuracy: 0.7988 - val_loss: 0.5326

Epoch 15/15

133/133 ————— 267s 2s/step - accuracy: 0.9062 - loss: 0.2190 - val_accuracy: 0.7915 - val_loss: 0.5560



Confusion Matrix:
[[1548 327]
[437 1438]]



TRIAL 3

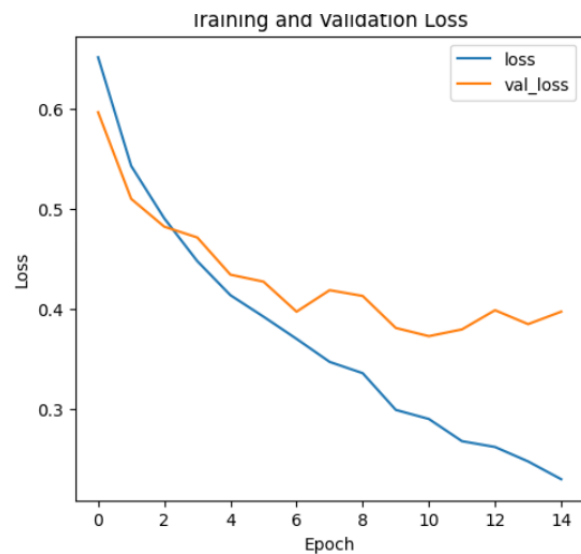
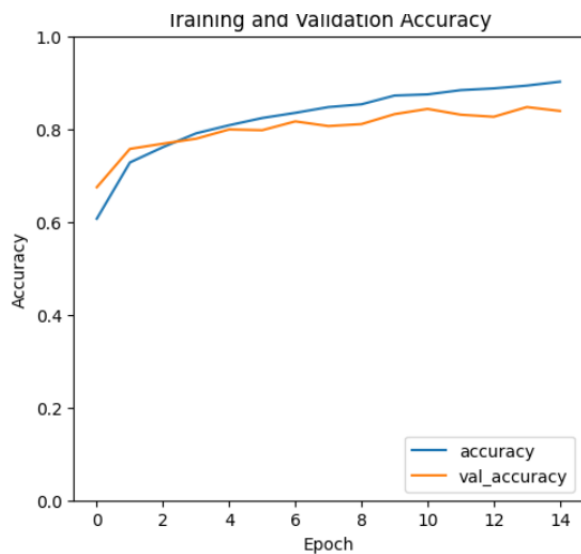
model.summary()

Model: "sequential"

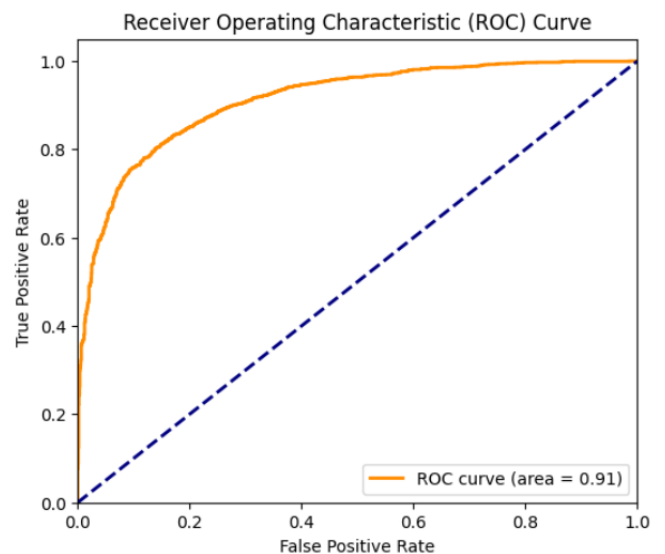
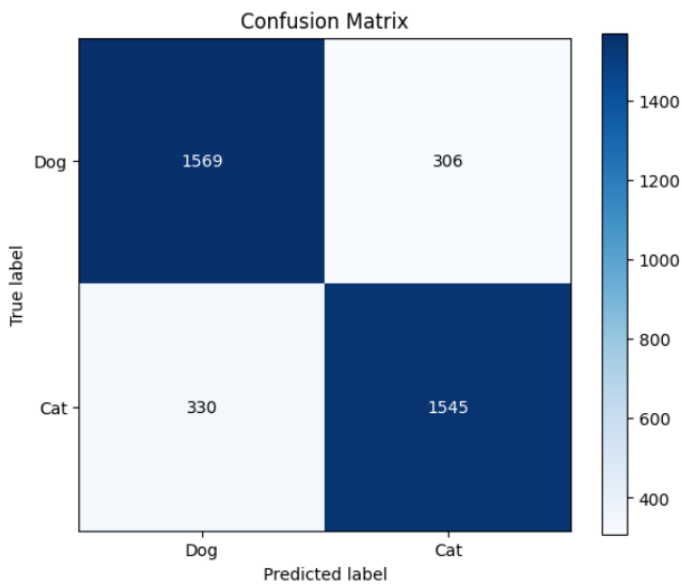
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	320
activation (Activation)	(None, 126, 126, 32)	0
max_pooling2d (MaxPooling2D)	(None, 42, 42, 32)	0
conv2d_1 (Conv2D)	(None, 40, 40, 32)	9,248
activation_1 (Activation)	(None, 40, 40, 32)	0
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_2 (Conv2D)	(None, 11, 11, 32)	9,248
activation_2 (Activation)	(None, 11, 11, 32)	0
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 32)	0
flatten (Flatten)	(None, 288)	0
dense (Dense)	(None, 32)	9,248
dense_1 (Dense)	(None, 32)	1,056
dense_2 (Dense)	(None, 1)	33
activation_3 (Activation)	(None, 1)	0

Total params: 29,153 (113.88 KB)
Trainable params: 29,153 (113.88 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/15
133/133 ————— 242s 2s/step - accuracy: 0.6068 - loss: 0.6515 - val_accuracy: 0.6746 - val_loss: 0.5966
Epoch 2/15
133/133 ————— 263s 2s/step - accuracy: 0.7280 - loss: 0.5429 - val_accuracy: 0.7572 - val_loss: 0.5100
Epoch 3/15
133/133 ————— 266s 2s/step - accuracy: 0.7610 - loss: 0.4906 - val_accuracy: 0.7685 - val_loss: 0.4822
Epoch 4/15
133/133 ————— 266s 2s/step - accuracy: 0.7907 - loss: 0.4477 - val_accuracy: 0.7791 - val_loss: 0.4714
Epoch 5/15
133/133 ————— 325s 2s/step - accuracy: 0.8081 - loss: 0.4137 - val_accuracy: 0.7991 - val_loss: 0.4342
Epoch 6/15
133/133 ————— 268s 2s/step - accuracy: 0.8237 - loss: 0.3923 - val_accuracy: 0.7974 - val_loss: 0.4273
Epoch 7/15
133/133 ————— 272s 2s/step - accuracy: 0.8348 - loss: 0.3701 - val_accuracy: 0.8165 - val_loss: 0.3973
Epoch 8/15
133/133 ————— 324s 2s/step - accuracy: 0.8472 - loss: 0.3471 - val_accuracy: 0.8064 - val_loss: 0.4188
Epoch 9/15
133/133 ————— 270s 2s/step - accuracy: 0.8532 - loss: 0.3358 - val_accuracy: 0.8106 - val_loss: 0.4130
Epoch 10/15
133/133 ————— 321s 2s/step - accuracy: 0.8721 - loss: 0.2991 - val_accuracy: 0.8320 - val_loss: 0.3810
Epoch 11/15
133/133 ————— 324s 2s/step - accuracy: 0.8746 - loss: 0.2900 - val_accuracy: 0.8433 - val_loss: 0.3728
Epoch 12/15
133/133 ————— 322s 2s/step - accuracy: 0.8837 - loss: 0.2678 - val_accuracy: 0.8308 - val_loss: 0.3795
Epoch 13/15
133/133 ————— 268s 2s/step - accuracy: 0.8875 - loss: 0.2620 - val_accuracy: 0.8264 - val_loss: 0.3987
Epoch 14/15
133/133 ————— 325s 2s/step - accuracy: 0.8935 - loss: 0.2476 - val_accuracy: 0.8473 - val_loss: 0.3848
Epoch 15/15
133/133 ————— 266s 2s/step - accuracy: 0.9019 - loss: 0.2298 - val_accuracy: 0.8388 - val_loss: 0.3973



118/118 — 17s 142ms/step



Receiver Operating Characteristic (ROC) Curve

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Across the three manual trials, increasing training epochs alone (Trial 1 to Trial 2, both using 2/64/64 filters and 2x2 pooling) did not improve generalization, since validation accuracy stayed near 79 percent in both runs while training accuracy climbed from 88.76 percent to 90.62 percent, widening the overfitting gap and pushing final validation loss up to 0.5560. Trial 3, which lowered the filter counts to a uniform 32 per convolution layer, enlarged the pooling window to 3x3, and reduced the dense layer size to 32 units, produced the strongest results of the set, reaching a validation accuracy of 83.88 percent (peaking at 84.73 percent on epoch 14) and the lowest validation loss recorded across all trials at 0.3973. The gap between training and validation accuracy was also narrowest in Trial 3, about 6.3 percentage points, compared to roughly 10 to 11 points in Trials 1 and 2, indicating the smaller, more aggressively pooled architecture generalized better rather than simply memorizing the training images. This suggests that for this dataset, reducing model capacity and increasing the pooling window size had a greater positive effect on validation performance than extending training time on an already large architecture. Overall, the manual tuning process shows that architectural adjustments, not epoch count alone, were the more effective lever for improving this CNN's ability to classify unseen dog and cat images.

Trial 4 attempted to scale the model back up to 64 filters per convolution layer while keeping the 5x5 kernel, 3x3 pooling, 64 dense units, and a batch size of 128, targeting a lower epoch count of 20 to test the minimum of the epoch range, while all training hyperparameters (binary_crossentropy loss, Adam at 0.001, no early stopping) stayed unchanged. This configuration repeatedly crashed the Google Colab session before a single epoch could complete, most likely because the combined memory and computation demand of three 64-filter layers, a 5x5 kernel, and a batch size of 128 exceeded what the free-tier runtime could handle, unlike the lighter 32-filter architecture in Trial 3 that trained successfully. As a result, no Training Accuracy, Validation Accuracy, or Dropout Rate data could be collected for this trial, and the manual tuning process had to stop here. This shows that for this dataset and hardware setup, model capacity is limited not just by what improves accuracy but by what the available compute can actually run, making Trial 3 the best result the experiment was able to produce.